

Proving the address, and the tap that need not be there

Two changes to the account a respondent is offered at the end of a Verto. One closes a hole that is already costing us mail we promised to send. The other is the extra tap on the Google door – a page that has to exist for a reason worth understanding, carrying a button that is labelled as the thing you just did.

Build 2026-09-15-a

5 boards + worked example + spec

Sibling of `../responder-accounts/`

The worked example – Priya, and the two doors she can walk through A – The two doors today, tap by tap B – What "unverified" costs: the silent account
C – The verification mail, and what the link opens D – The end card and `/you` , with the address proved E – The Google door, one tap shorter
Spec – §1–§9, the cuts, and what each piece costs

The worked example

Same fiction as the two decks before this one. **Haverley Town Council** runs **Car-free High Street**; Priya finishes it on a phone on the bus and is offered an account. Everything below is drawn at the copy that is in `config/locales/en.yml` today, not at invented copy – where a board proposes new words they are marked as such.

The card she sees	Keep this · "Keep your Vertos" · "Pick a password and you're in. You'll see your answers next to everyone else's, and hear what happens next." · Create my account <code>player.join_*</code>
Door one	Address and a password. Twelve characters minimum, the same floor <code>User</code> has. 3 taps
Door two	Continue with Google. 4 taps
What the two doors leave behind	Door two stamps <code>email_verified_at</code> ; Google has asserted the address. Door one never does. Two later paths can still rescue it – both undiscoverable, neither offered as a fix (board B). <code>the hole</code>
What that costs	<code>PlayerAudience.for_survey</code> filters on <code>where.not(email_verified_at: nil)</code> . Priya is in the account, holds the claim, and is excluded from every mail the council ever sends about it – including the one the card promised her. §2

Door one — address and password 3 TAPS + TYPING

The card opens collapsed on its pitch and one button. The form behind it is 560px of an 844px screen, which is why it is folded (d56a8f0 , 14 Sep).

End screen /play/:token

Thanks for taking part!

KEEP THIS

Keep your Vertos

Pick a password and you're in. You'll see your answers next to everyone else's, and hear what happens next.

Create my account

The card is #272D4A — Playverto's surface, not the creator's.

The form, revealed same page

Continue with Google

OR

priya.n@gmail.com

..... Show

At least 12 characters. You'll use it to sign back in on another device.

Create my account

You're signed in the moment you're done — nothing to wait for in your inbox.

The hand-back /you/sign-in/:token

playverto

You're all set

Tap below to open your account and keep what you've played.

Sign me in

Links work once, and expire 20 minutes after they're sent.

Signed in /you

You're signed in.

priya.n@gmail.com

3 No Verified tag, because nothing proved the address. email_verified_at is NULL and stays that way.

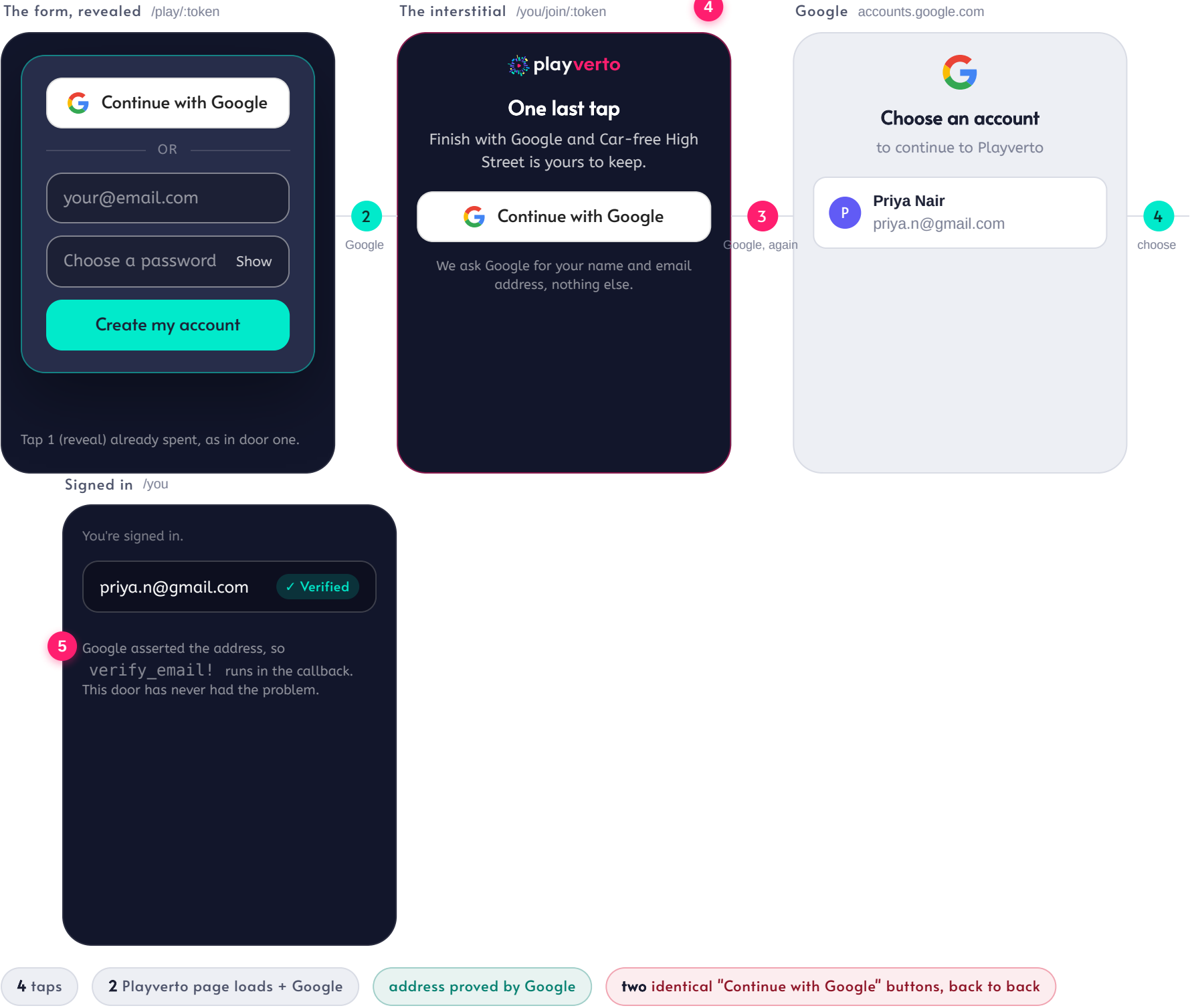
3 taps

2 page loads after the card

address never proved

Door two — Continue with Google 4 TAPS

Two taps here could each be called "the extra step", so both are named. Tap 1 is the reveal: the card opens collapsed, so "Continue with Google" is not on screen until "Create my account" is pressed — added deliberately on 14 Sep (d56a8f0) because the open form was 560px of an 844px screen. It is a real cost and it is shared with the password door. Tap 3 is the interstitial, which is unique to Google and is a second button identical to the one just pressed. This deck treats tap 3 as the extra step, because tap 1 buys something measurable and tap 3 buys nothing the respondent can see — but if the brief meant the reveal, §6 is the wrong section and the answer is to surface Google above the fold on the collapsed card, which is a layout change and a different piece of work.



NOTES

- 1 The fine print already promises there is no email. "You're signed in the moment you're done — nothing to wait for in your inbox." That sentence is true today and board C makes it false, so it is one of the strings that has to change in the same commit — in all 26 locale files. §5 §7

2 Door one has an interstitial too, and it is not the same problem. PlayerSignInsController splits GET from POST because corporate link scanners follow GETs, and a single-use link a scanner can spend is a link the recipient can never use. That reasoning is about emailed links. A signup link is handed straight back in the JSON response and never enters an inbox — so the scanner argument does not cover this screen, and §6 asks whether it should auto-submit. It also carries a copy bug worth a one-line fix while someone is in there: the heading and body switch on proves_address? , but the fine print below them does not — so a signup arrival is told "Links work once, and expire 20 minutes after they're sent", describing an email that was never sent. §6
- 3 The end state of door one is an account nobody can write to. Board B is this fact drawn out. And the account page offers her nothing: /you/account renders the Verified tag when she is verified and renders nothing at all when she is not — no explanation, no resend. Two routes out do exist, and board B shows why neither counts. §2 §4

4 This is the screen the brief is about. It is titled "One last tap", which is the app apologising for itself. It exists for a real reason — OmniAuth 2.x makes the request phase a CSRF-protected POST, and /play/:token is inside the Service Worker's scope and can be served from cache with an authenticity token of any age, so the card cannot POST to Google itself. §6 separates that constraint (which is real) from the decision not to auto-submit (which is a UX judgement, and reversible). §6
- 5 Worth saying plainly: the Google door is the better door on every axis except tap count — no password to invent at the end of a survey, and an address that arrives already proved. If only one of the two changes in this deck ships, §6 is the one that compounds, because it moves people onto the door that does not need §1 at all. §6

Neither flow is broken. Door one produces a working, signed-in account that can play, claim and read results — it simply produces one the product may never mail. That is why this is a gap rather than a bug, and why it has survived.

I · WHAT THE CARD PROMISED

KEEP THIS

1

Keep your Vertos

Pick a password and you're in. You'll see your answers next to everyone else's, and **hear what happens next.**

Four words doing the work: "hear what happens next" is the reason a respondent gives an address rather than just closing the tab. It is the only part of the pitch that needs mail.

2 · WHAT THE COUNCIL CAN ACTUALLY SEND HER

PlayerAudience.for_survey

2

kept = players who claimed it — Priya is in
...where.not(email_verified_at: nil) — **Priya is out**
...minus this org's opt-outs
...minus global suppressions

Nothing. Not the results, not the follow-up, not the next Verto.

`deliverable?` applies the same rule one player at a time, so there is no second path that leaks past the scope. The filter is right — an unproven address should never be mailed. The bug is that door one produces unproven addresses and calls it a finished signup.

3 · AND THE ACCOUNT OFFERS HER NOTHING

/you/account — as it renders today

Email

3

priya.n@gmail.com

This is where your Vertos are kept.

The template renders a `✓ Verified` tag when `@player.email_verified?` and renders nothing in the unverified case — no tag, no sentence, no resend. From inside the account there is no evidence anything is wrong and no control that would fix it.

4 · TWO ACCIDENTAL CURES

Sign in to your Vertos

Email me a link instead

4

priya.n@gmail.com

Email me a link

Two routes do exist, and that changes the framing: the capability is there, the discoverability is not. (1) Sign out and ask for an emailed link — that mints `ORIGIN_EMAIL`, and spending it runs `verify_email!`. (2) Come back later and tap Continue with Google at the same address: `locate_or_create_player!` falls back to `Player.find_by(email_address:)`, finds the existing password row, and `verify_email!` runs unconditionally on it.

Neither is signposted anywhere, and route 1 requires signing out of a working account. So the work is a purpose-built token and a place to tap it — not a missing capability.

NOTES

- 1

The three sentences on the card are the creator's to overwrite (`Survey#join_body_text` falls back to this copy when the field is blank), so the promise varies per Verto — but the default is the one most Vertos ship with, and it promises mail.
- 2

This is the whole business case and it is worth stating without decoration. Every respondent who takes door one is a respondent the creator paid to acquire, holds an account for, and can never contact. It fails silently in both directions: nothing errors, no job retries, the creator's audience count is simply smaller than their claim count and nobody has asked why. §2

- 3

Worth being precise about who is affected. Google respondents are fine — the callback stamps `verify_email!` because the provider asserted the address, which is strictly better proof than an emailed link. Emailed-link respondents are fine. It is exactly the password door, which is the newest of the three and the one the card leads with. §1

- 4

So there are two defensible fixes and this deck picks one. Either send a confirmation mail (§1) or widen what counts as proof — but the second is not available: a typed address plus a password proves somebody chose a string, not that they can read the inbox, and treating it as proof would mean mailing whatever a stranger typed. §3 works through why the gate stays where it is. §1 §3

Not in scope, and worth saying so: this deck does not propose blocking an unverified respondent from anything they can do today. They keep the account, the claims, the results and the wallet. The only thing verification unlocks is being written to, which is the only thing it is evidence for.

WHAT THE LINK OPENS

1



2

3

4

I · THE CARD'S FINE PRINT

TODAY

Create my account

You're signed in the moment you're done – **nothing to wait for in your inbox**. We never post anything, and you can delete the account in one tap.

PROPOSED

Create my account

You're signed in the moment you're done. We'll send one email to confirm the address, so the organisation can tell you what happens next. We never post anything, and you can delete the account in one tap.

The current sentence is the one thing in the product that **actively contradicts** this change, and it is the reason the copy edit is not optional dressing – it ships in the same commit or the card lies. 26 locale files.

4 · WHAT THE CREATOR SEES

Audience for this Verto

Kept this Verto

412

Can be emailed

388

Waiting on confirmation

24

Not proposed here, and noted so nobody assumes it. Today the creator sees an audience count smaller than their claim count with no explanation for the gap. Naming the gap is the natural follow-up to this work, but it is a separate change on a different surface and it is costed separately in §9.

2 · /YOU , ADDRESS NOT YET PROVED

playverto

Confirm your email

We sent a link to priya.n@gmail.com. Until you tap it, Haverley Town Council can't tell you what happens next.
[Send it again](#)

Your Vertos

Amber, not red – the account works and nothing is blocked. It cannot reuse the creator's banner: `shared/_email_confirmation_banner` opens `if Current.user && ...`, so it is invisible to a respondent by construction. This is a new partial, drawn in the chrome `/you` already owns – the full-bleed `.you-beta` strip above the bar, and the `.you-flash` pill inside `.you-main`.

3 · /YOU/ACCOUNT — THE EMAIL ROW

TODAY, UNVERIFIED

priya.n@gmail.com

PROPOSED, UNVERIFIED

priya.n@gmail.com

Not confirmed

[Send the link again](#)

CONFIRMED

priya.n@gmail.com

✓ Verified

One `<% else %>` on an `if` that already exists. The template renders the teal tag when verified and nothing at all otherwise; the amber tag fills that branch and the state stops being invisible.

NOTES

- 1

Three sentences, one of which is new. It has to say **what confirming buys** rather than that an email is coming – "you'll get an email" is a cost with no stated benefit, and the card is asking for the address at the exact moment a respondent is deciding whether to close the tab. §5 §7
- 2

The banner renders on `/you` only, not on the end screen. The end screen belongs to the creator's Verto and already carries the card; putting a Playverto system banner there would be the third thing competing for the same moment. Note it is a **new partial rather than a reused one** – the creator's equivalent is gated on `Current.user` and a respondent is not one, which is a small cost the estimate carries rather than a blocker. §4
- 3

The **resend is the part that needs care**. Rate-limited like the creator's (5 per 10 minutes) and reachable only from inside a signed-in session, so it can only ever mail the address of the account you are already in – which is what keeps it from being an enumeration oracle or a mail-bomb button. Note this is a different threat model from `#join`: that endpoint takes an address from an anonymous caller, this one takes none at all. §4
- 4

Sequencing that matters: the Google door already stamps `email_verified_at`, so the moment the amber tag exists it is correct for Google respondents without any further work. The tag is not new state – it is the state the database has been carrying all along, finally drawn. §2
- 5

One number worth having before this ships, and it is a one-line query rather than a project: `Player.where(email_verified_at: nil).where.not(password_digest: nil).count` – every respondent already in this position. They are the backfill question in §8, and whether that is 20 people or 2,000 changes whether a one-off mail-out is worth writing. §8

Explicitly out of scope: gating anything on confirmation. No blocked claim, no hidden results, no locked wallet. The creator gate for `User` is on publishing because an unproven creator can collect other people's birth dates under a borrowed address; an unproven respondent can only fail to hear back, so the consequence and the remedy are the same thing and a gate would add nothing but harm to conversion.

The contrast that makes this worth doing

SAME STRATEGY, SAME RESPONDENT

Both of these buttons start `google_player` . One of them needs an interstitial and the other does not, and the difference is not the respondent — it is which page the button is standing on.

From the end card /play/:token

1

Continue with Google

Page is **shared-cached server-side** (link + deck + locale, 1h) **and** cached again by the Service Worker.
→ its CSRF token was minted during **someone else's request**.
→ cannot POST to `/auth/google_player` .
Needs the interstitial.

From the sign-in page /you/sign-in

2

Continue with Google

Outside the worker's scope, rendered fresh per request.
→ token is **this browser's, this request's**.
→ `button_to` posts straight out. **One tap**.

And the creator's /session/new

Continue with Google

Same `button_to` , same reasoning in its own header comment, one tap. The Service Worker's scope was deliberately narrowed to `/play/` to keep it off the studio.

Three ways out — and which one to do first

2, THEN MAYBE 1, PROBABLY NEVER 3

I · Auto-submit the interstitial

SECOND

Keep the page, keep the live token, keep the handoff. The form submits itself on first view, so the respondent sees a branded "Taking you to Google..." beat instead of a button that looks exactly like the one they just pressed.

Removes: the tap. **Keeps:** the page load.

Costs: the Back-button trap the controller's comment names — Back from Google re-GETs this page and would re-fire. Guarded by refusing to auto-submit on a `back_forward` navigation, which is one small Stimulus controller — the layout already loads importmap through `layouts/_head` , and `sw_register.js` already listens on `pageshow` , so neither the plumbing nor the idiom is new. The button stays in place as the fallback for no-JS and for the guarded case.

Risk: low. No CSRF, OmniAuth or cache change. The button stays; only who presses it moves.

Why second, not first: it removes a tap, but the tap is only *felt* because of the label (option 2). Fix the label, see whether the step still reads as friction, then decide. The guard is also the part most likely to be got wrong — and the page has a fourth job nobody's docstring mentions: `redirect_to you_path if player_signed_in?` , which is where the data-loss bug below lives. Its test cost is three new cases rather than a rewrite (§10).

2 · Relabel the second button

RECOMMENDED · DO THIS FIRST

The two buttons render the same `I18n` string. `auth.social_continue` draws the button on the card *and* the button on the interstitial, so a respondent presses "Continue with Google" and lands on a page whose button is also called "Continue with Google". It reads as a no-op because it is labelled as one. Give the second one its own key — a continuation, not a repeat of the choice already made.

Removes: the *feeling* of a wasted tap, which is what the brief actually reported.

Costs: one new key in 26 locale files, one view line.

Risk: none — no controller, no CSRF, no test rewrite. It is the smallest change in this deck and it addresses the complaint as reported.

3 · Delete the page

LARGEST

Scope

`OmniAuth.config.request_validation_phase` to accept the single-use `Player0authHandoff` token for `/auth/google_player` and delegate to `TokenVerifier` for everything else. The card then posts once, straight out.

Removes: the tap *and* the page load.

Costs: a hand-written replacement for a global security hook that also guards creator sign-in, branching on `PATH_INFO` . The handoff has the right properties for the job — HMAC-SHA256, 32 random bytes, 15-minute expiry, atomic single-use — but the session write that `/you/join` performs has to move to OmniAuth's `origin_param` , and the middleware is outside the controller so it can still write a real session.

Risk: high, and concentrated in the one place a mistake is worst. Not recommended as a first move; reachable later if the page load still matters once the tap is gone.

Three live bugs on this path, found while reading it

INDEPENDENT OF THE ABOVE

In an embedded Verto, neither door can finish

BOTH PATHS · VERIFIED

A Verto is routinely framed by a third party — the card has its own copy for it (`join_embedded` , "we can't see what you played elsewhere"), and `_embedded()` tests `window.self !== window.top` to decide whether to show it. So the app knows when it is in a frame and adapts. **What it does next cannot work there.**

`after_action :allow_embedding` is scoped to `show` , `test_show` and `live_test_show` — the player pages only. The global policy is `frame_ancestors :self` . But both doors finish by navigating *in-frame*: `_joinDone` calls `window.location.assign(next)` , and `next` is `/you/sign-in/:token` or `/you/join/:token` — neither of which allows framing. The third-party parent gets a refused frame where the account was meant to appear. The `SameSite=Lax` cookie the code already warns about would not have survived the trip either.

So on an embedded Verto nobody can create an account at all, by any route, and `test/integration/player_embedding_test.rb` contains the word "join" zero times. The fix is to break out of the frame for that one navigation (`window.top.location` , or `target="_blank"`) rather than to widen the CSP on the account pages — which is the more tempting and much worse option.

The already-signed-in respondent loses the run

DATA LOSS · REPRODUCED

`PlayerJoinsController#show` returns `redirect_to you_path` if `player_signed_in?` **before** it writes `session[:player_oauth_handoff_id]` . Because the `/play` HTML is shared-cached it cannot vary on sign-in state, and neither the card nor the JS gates on it — so a respondent who is *already signed in*, finishes a Verto and taps Continue with Google mints a handoff, is bounced to `/you` , and **the claims for the run they just finished are never applied**. The handoff expires unspent 15 minutes later.

Reproduced, not inferred. A throwaway integration test — sign a respondent in with an `ORIGIN_EMAIL` link, complete a Verto, `POST /play/:token/join_google` , then `GET` the URL it hands back — holds all four assertions: the handoff is minted *with* the run's claims, the respondent is redirected to `/you` , `session[:player_oauth_handoff_id]` is never written, `consumed_at` stays nil, and `PlayerClaim` count is unchanged. 12 assertions, 0 failures, 1.6s.

The email path does not have this hole: `PlayerSignInsController#create` applies the claim payload unconditionally. Fix is to spend the handoff and apply the payload before redirecting.

The venue lever does not reach this page

EVENT-DAY

`#join_google` is capped at `15 × PLAYER_JOIN_RATE_LIMIT_SCALE` per IP per 5 minutes — the lever added on 14 Sep so a room full of people on one venue NAT can all make accounts. `PlayerJoinsController` 's own gate is `30` per 5 minutes and is **not scaled by anything**. At `SCALE = 1` the first gate binds; at `SCALE ≥ 3` the unscaled interstitial becomes the binding constraint on exactly the crowd the lever exists for.

Worse, its refusal is `redirect_to you_path` with a "too many attempts" alert — a signed-out account page, not a message on the card they were standing on. Fix is one multiplication.

- NOTES
- 1

The controller's own comment understates its case. It says the page "can be served from cache with an authenticity token of any age", which sounds like a problem for offline respondents only. The stronger fact is in `cached_play_page` : the HTML is cached as *shared bytes* keyed on link + deck version + locale, so for nearly every respondent the token in that page was minted during a stranger's request. A direct POST would fail for almost everyone, not for a slow-connection minority — which makes the interstitial more load-bearing than its own docstring claims, and option 3 correspondingly more serious. §6
- 2

This is the fact worth carrying out of the whole board: **the extra step is not the price of being a respondent, and not the price of the google_player strategy**. It is the price of the button standing on a page that is cached twice over. The respondent's own sign-in page runs the identical strategy in one tap. §6
- 3

And the sharpest version of the complaint is a **two-line fix, not an architectural one**. `_join_block.html.erb` and `player_joins/show.html.erb` both draw their button from `t("auth.social_continue")` — the same string, twice, seconds apart. The page is not badly placed so much as badly labelled: it announces itself as the thing you just did. That is why option 2 goes first and option 1 waits to see whether anything is left to fix. §6

Rejected without further work: making `/play/:token` per-session so its token is live. The shared cache exists because rendering that page is a large share of a journey's CPU, and trading it for one tap would be paying in the thing that carries a venue full of respondents. Also rejected: allowing `GET` on the OmniAuth request phase — the config is global, so it would re-open creator sign-in to the cross-site initiation class that OmniAuth 2.0 closed.

Spec

What each board implies in code, what it costs, and the things this deck deliberately does not do. Everything below was read out of the tree at 817d3a9 (14 Sep 19:31); line references are to that commit.

1 The token, and why there is no migration.
`Player` already carries `email_verified_at` and an idempotent `verify_email!` that only writes when the column is nil. What it lacks is a token purpose and a send. Add `generates_token_for :email_confirmation, expires_in: 7.days { email_address },` mirroring `User` line-for-line — keyed on the address, so that changing the address invalidates any outstanding link, which is the property that makes a stale token harmless.

7 days, not the 20 minutes a sign-in link gets. The two are different kinds of thing: a `PlayerSignInLink` is a bearer credential stored as a digest in its own table, and is sized like one. This token grants nothing — spending it sets a timestamp — so it is sized for how long a real inbox takes to get read. **No migration, no new table, no schema change.**

One document has to move with it. `docs/DATA_RETENTION.md` states as property 4 that "every credential is single-use, a digest, or somebody else's", and enumerates the three. A `generates_token_for` token is *none* of them — it is reusable until it expires, never stored, and ours. That is the right trade for a token that grants nothing, but the property as written becomes false and has to be rewritten rather than quietly outgrown.

3 Why nothing else gets gated.
The creator gate sits on **publishing** because an unproven creator can collect other people's birth dates and locations under an address its owner never consented to. The respondent case has no equivalent: an unproven respondent holds only their own answers. The only thing their address is evidence *for* is whether we may write to it, so that is the only thing it gates.

Claims, results, the wallet, comparison, the account itself — all stay open. A gate on any of them would cost conversion on the exact card that exists to win it, and would protect nothing. **CLAUDE.md's note that SMTP is optional in this deployment is the second reason:** a gate on anything load-bearing would mean a misconfigured mail server locking respondents out of their own answers.

5 Copy the respondent mailer, not the creator's.
`PlayerSignInMailer` is the file to copy, and specifically *not* `EmailConfirmationsMailer` — the creator's is older and hardcodes English with no `I18n.with_locale`. The respondent one already carries the two things this mail needs:
`I18n.with_locale(SupportedLocales.coerce(player.preferred_locale))`, because a mailer runs in a Solid Queue job with no request and therefore no `Current.locale` — without it the mail renders in whatever locale the worker was last left in. And `headers["X-Entity-Ref-ID"] = SecureRandom.uuid`, so a resend does not stack under the original in a mail client and get missed.

Subject names the **organisation**. HTML and text parts both. Delivery wrapped the way `EmailConfirmationsController.deliver` wraps it — `rescue`, report, return nil — because the account has already been created and persisted by the time this runs, and a mail failure must never turn a completed signup into an error.

Two calls a builder will hit, settled here. *Honour* `EmailSuppression`. The sibling transactional mail does not check it, but that table carries hard bounces and complaints — sending a confirmation to an address that has already bounced achieves nothing and spends sender reputation to do it. Skip the send, leave the account unverified, and let the banner say so. *No welcome mail on confirmation.* The creator flow sends one; a respondent has had the account since the end of the Verto and the landing page already says what changed, so a second mail would be a notification about a notification.

And it must degrade when SMTP is unconfigured, which this deployment allows. The pattern already exists: `MailConfigCheck.deliverable?` (`lib/`, not `app/lib/`), used at `PlayerSessionsController` — when mail cannot go out, do not mint the token and do not promise the mail. The card's new sentence has to be conditional on the same check, or it promises an email that a misconfigured deploy will never send.

And it must carry `?locale=`, which no emailed link does today.
`#join_google` already puts it in the URL on purpose — "where `resolve_locale's ?locale=` already means the pages in between are drawn in it too". `#join` and `PlayerSignInMailer` do not. So a mail rendered in the respondent's language currently opens a page in the browser's, and a verification mail that copies the mailer verbatim inherits that. Put the locale in the link.

With a trap in the testing of it. `deliverable?` opens `return true` unless `ActionMailer::Base.delivery_method == :smtp`, and `delivery_method` is `:test` in test and not `:smtp` in development — so it is unconditionally true in both, and the pessimistic branch is unreachable locally without stubbing.
`test/integration/player_password_sign_in_test.rb` already stubs it exactly for this, and the new copy's test copies that.

7 The 26-locale obligation, and what is already paid for.
New UI strings go in `en.yml` plus the other 24 hand-written files, then `bin/rails i18n:en_us` regenerates `en-US.yml`; `LocaleEnUsTest` fails if it is stale. Put the new keys under `you.*`, which is already covered by `locale_structure_parity_test.rb`, rather than inventing a namespace. Anything JavaScript reads has to go through `window.I18N` — `js:` plus the curated slice in `_i18n_js.html.erb` — or it renders as a raw dotted key.

Some of it is already paid for, but check before counting on it. Three keys under `js.player.*` survive from the emailed-link era and are translated in all 26 files: `join_sent_title` ("Check your inbox"), `join_sent_body` and `join_sent_again` ("Send it again"). Two of the three reuse verbatim — the title and the resend label are exactly the words a confirmation banner wants. **The third does not:** `join_sent_body` reads "We've sent a link to %(email). It signs you in and keeps this Verto — no password needed", which describes the sign-in link this flow no longer sends. It needs new words in all 26, so count it as new. None of the three is referenced in the JS any more, so they are dead keys being revived rather than shared.

For a session with no `ANTHROPIC_API_KEY`, `bin/rails i18n:translate` takes `PRINT= / TRANSLATIONS=`. Expect **+N lines, 0 deletions** per file; a deletion means stop.

One thing that has bitten before: a new mailer namespace has to be added to `NAMESPACES` in `test/lib/locale_respondent_parity_test.rb` *in the same commit that creates it*. That file's own header records that forgetting it hid an English-only feature for three pushes.

2 The gate already exists and is already firing.
This is the fact that makes the whole change small. `PlayerAudience.for_survey` has filtered on `where.not(email_verified_at: nil)` since it was written, and `deliverable?` applies the same rule per-send so nothing leaks past the scope. Nothing needs building to *enforce* verification. The gate is correct, it is live, and it is silently excluding every password-signup respondent in the database.

So §1 is not "add a gate". It is "give the people already behind the gate a way through it".

4 The resend, and not building an oracle.
The resend lives on `/you` and `/you/account`, behind a signed-in player session, and takes **no address parameter** — it can only ever mail the account you are already inside. That is what keeps it from being either an enumeration oracle or a mail-bomb button, and it is a materially different threat model from `PlayerController#join`, which accepts an address from an anonymous caller.

Rate-limited at 5 per 10 minutes, matching the creator's. **The confirm landing page carries no resend** and echoes no address beyond what the token resolves.

6 The Google door: what is constraint and what is choice.
Constraint (real, not negotiable cheaply). OmniAuth's request phase is a CSRF-protected POST — `allowed_request_methods = [:post]` plus `request_validation_phase = TokenVerifier.new` — and Rails 8.1 binds the CSRF token to the session. The card cannot produce a token that validates, for a reason stronger than the controller's own comment gives: `cached_play_page` caches the `/play/:token` HTML as **shared bytes** keyed on link + deck version + locale, so for most respondents that page's token was minted during a stranger's request. The Service Worker cache is a second, independent reason. A direct POST would fail for nearly everyone.

Choice (reversible). That the interstitial does not auto-submit is a separate decision, stated as such in the controller: a Back-button trap, and a dislike of a GET that starts a redirect when a scanner touches it. The scanner half is weak here by the controller's own later admission — "nothing is spent here", and a test asserts it. **The Back-button trap is real** and is what the auto-submit has to guard against.

And there is a third thing, which is neither constraint nor architecture: the two buttons are the same string. `_join_block.html.erb` and `player_joins/show.html.erb` both render `t("auth.social_continue")`, so the respondent presses "Continue with Google" and arrives at a page whose button is also "Continue with Google". The step reads as a no-op because it is labelled as one.

Recommended, in this order: **(1) give the second button its own key** — one new string, one view line, no controller and no test rewrite, and it addresses the complaint as reported. **(2) Then, if the step still reads as friction, auto-submit**, guarded against `back_forward` navigations with the button left in place for that case and for no-JS. Doing it in that order means the risky half is only paid for if the cheap half does not settle it.

One correction worth recording, because it narrows option 3. "The handoff response cannot carry a live CSRF token" is true of today's wiring, not of what is possible: a token minted on an earlier request in the *same browser* does validate on a later POST to `/auth/google_player`. What blocks it is one line — `:join_google` sitting in the `null_session` list, which nulls the session and drops the `Set-Cookie`. So a fourth path exists: lift `join_google` out of `null_session`, let it establish a session and hand back a live token, and let the card post straight out. It is cheaper than rewriting the validation phase — but it fights the two reasons that action is cookie-free (the page is service-worker cached and is routinely framed by a third party, where a `SameSite=Lax` cookie is not sent), so it is a narrowing of option 3, not an escape from it.

Deleting the page (option 3 on board E) means hand-writing a replacement for a global security hook that also guards creator sign-in. It is reachable and it is not first. Note also a fourth job the page does that its own header does not mention: `return redirect_to you_path if player_signed_in?` — which is where P3's data loss lives, and which anything removing the page would have to re-home.

8 The respondents already in this state.
One query before shipping, because it decides whether a second piece of work exists: `Player.where(email_verified_at: nil).where.not(password_digest: nil).count`. Every one of those is an account that was promised mail and cannot receive it.

If it is small, do nothing — the banner on `/you` catches them the next time they visit. If it is large, a one-off mail-out is a separate, gated change with its own rate shaping, **not something to bolt onto this push:** sending to thousands of addresses that have never received mail from us is exactly how a domain lands on a suppression list.

9 What this deck does not do.

One thing it does unlock, and the code says so itself. `/you/account` refuses to let a respondent change their address, and gives the reason in as many words: doing so "would need the same proof-of-inbox the sign-in link gives – which is the thing this page cannot send". After §1 it can. The email-change flow is not in this deck and is not costed in the table, but it stops being blocked, and that comment becomes false and has to be rewritten with the rest.

It does not gate anything (§3). It does not touch the Service Worker, so no `CACHE_VERSION` bump is needed – the rule is that content and markup reach respondents on the next ordinary online visit, and only a change to the worker's *own behaviour* needs the bump. It does not change `cached_play_page`. It does not build the creator-facing "24 waiting on confirmation" panel that board D sketches – that is real and it is costed below, but it is a different surface. And it does not merge the verification mail with a sign-in link: they expire differently, prove different things, and one of them is a credential.

11 The lever that would quietly become a mail-bomb amplifier.

This is the one thing in the deck that is not optional, and it is the reason P1's risk is not "low". `PLAYER_JOIN_RATE_LIMIT_SCALE` was added on 14 September so a room of 250 people on one venue NAT could all make accounts. Its own comment says exactly why that was safe: the caps "were sized against a threat that no longer exists... it doesn't send mail and hasn't since the emailed-link era", and *"that is why the per-IP half of them can be scaled for a crowd, which a mail-bomb guard could not be."*

Sending a confirmation from `#join` falsifies that sentence. It turns a documented non-mailing endpoint back into a write primitive against a stranger's inbox, and one env var – clamped at `(1, 10_000)` – into the multiplier on how many strangers one IP may write to.

`MAX_JOIN_ADDRESSES_PER_IP` is `30 × SCALE` per hour *per published Verto*; at the default that is 30 inboxes an hour, and at an event setting it is not.

What bounds it on the other side, and should be stated in the code: the mail fires on exactly two of `#join`'s branches – a genuinely new `Player`, and an adoptable shell being given a password. It must *not* fire when an existing account authenticates, when the answer is `use_google`, or when the password is wrong. So a stranger's address can be mailed once for an account that did not exist, and never again by the same route once it does; the exposure is first-contact only, which is what makes an unscaled per-IP counter sufficient rather than a per-mailbox one.

The fix is small and it has to ship with P1: bound the mail on its own counter at the **unscaled** number. The lever's job is to let a crowd create accounts; it was never a statement about how much mail one address may send. Creating an account and mailing a stranger are different acts and want different bounds.

And the prose has to move with the code, which in this repo is not a nicety. Two comments become false the moment this ships and both have to be rewritten in the same commit. `player_controller.rb`'s rate-limit header says, in as many words, "An earlier version of this comment justified all of that as protection for an inbox – join 'SENDS MAIL to an address a stranger typed'. *It does not*" – a comment that already records being wrong once about exactly this. And the paragraph under it, which concludes "that is also why the per-IP half of them can be scaled for a room full of people, which a mail-bomb guard could not be", is the reasoning the new counter exists to replace. Leaving either standing would hand the next reader a justification for widening the very cap that now guards an inbox.

And there is no feedback loop to catch it going wrong.

`EmailSuppression::REASONS` declares `hard_bounce` and `complaint`, but nothing in the app ever writes them – `record!` is called only from the two unsubscribe controllers, always with `"unsubscribe"`, and there is no ESP webhook route anywhere. So a send at join-card volume to addresses nobody has proved would bounce into silence, with no mechanism to stop trying. That is an argument for the unscaled counter, not against the feature – but it should be a known gap rather than a discovered one.

A second comment joins the list above: `db/migrate/..._create_players.rb` says of the column, "Until then the address is unproven, and *nothing may be sent to it beyond the sign-in link itself*." That file already carries one claim this repo has since falsified; a second should be corrected in the same commit rather than left to accumulate.

Two more facts worth having while sizing that counter. The per-address bounds – `join_email` at 5/20min and `MAX_JOIN_PER_ADDRESS` at 5/hour – key on the **exact lowercased string**, so `victim+1@gmail.com` and `victim@gmail.com` are different budgets and the same mailbox. And `code_budget_ok?` keys include `@survey.id`, so "5 per address per hour" is five *per published Verto* per hour. Neither is a flaw today, because nothing is being sent; both become the shape of the exposure the moment something is.

What it costs, and when it is live

Sized against `e29cb2d` (14 Sep) – the whole respondent-Google feature, 60 files and 1,931 insertions including two migrations and 26 locale files, built and shipped in one session. That is the yardstick this repo actually moves at. Every row includes its own `bin/gate` run: five checks, ~7½ minutes, and roughly a one-in-three chance of losing the race to another session and re-running. **A green push to Main deploys itself** – CI builds the image, the `deploy` job moves `:main` and POSTs Render's hook – so "live" is the push, not a separate release.

PACKAGE	FILES	BUILD	RISK	LIVE
P1 · Verification mail token, mailer + 2 views, controller, 2 routes, send from <code>#join</code> , <code>/you</code> banner, account tag + resend, card copy	~38 26 of them locales	¾ – 1½ days	Medium , and §11 is why. No migration and no schema change, and the gate it feeds already exists – but turning mail back on at <code>#join</code> re-arms a vector the repo deliberately disarmed, so the unscaled mail counter ships with it or not at all.	Same day
P2a · Relabel the second button its own key instead of <code>auth.social_continue</code> twice (board E, option 2)	~28 26 of them locales	~1 hr	None. One string and one view line. The smallest change here, and it addresses the complaint as reported.	Same day
P2b · Guarded auto-submit only if the step still reads as friction after P2a (board E, option 1)	3 + 3 tests	2 – 3 hrs	Low, but not zero: the <code>back_forward</code> guard and the expired-handoff case are both easy to get wrong. Deferred on purpose.	Optional
P3 · Bug: embedded Vertos cannot finish signup break out of the frame for the one navigation that leaves <code>/play/</code>	2 + tests	2 – 3 hrs	Low to fix, high to have : on an embedded Verto <i>no</i> respondent can create an account by any route today, and no test covers it. Ship with P4 and P5.	Same day
P4 · Bug: signed-in respondent loses the run spend the handoff and apply claims before the redirect	2 + test	~1 hr	Low, and it is live data loss today – this is the row to ship first.	Same day

Playverto · Proving the address, and the trap that need not be there

10 The test surface, and one guard that would quietly stop guarding.

These flows carry roughly **96 integration, unit and mailer tests across 7 files**, plus **~31 browser tests across 3 system files**. There are no fixture YAMLS anywhere in the repo – every test builds its own Organisation, Survey and Player inline with `SecureRandom`-suffixed slugs, so a new test copies that shape rather than adding a fixture.

The seven: `join_prompt` (19), `player_claims` (21), `player_google_join` (20), `player_password_sign_in` (12), `player_sign_in` (10), `player_sign_in_mailer` (7), `player_oauth_handoff` (7). The three browser files are the expensive ones, since the system suite is the ~5-minute half of the gate: `end_screen_layout` (20), `password_visibility` (7), `end_screen_contrast` (4).

The trap. `test/integration/player_claims_test.rb` guards the "join sends no mail" property with `assert_no_difference -> { ActionMailer::Base.deliveries.size }`. The test environment uses the `:test` queue adapter, so a `deliver_later` mail never touches `deliveries` – that assertion would go on passing while the behaviour it describes was reversed. **It has to become** `assert_enqueued_emails` **in the same commit**, or the suite will report green about a fact that is no longer true. The sibling assertion on `PlayerSignInLink.last.origin` needs the same read.

Auto-submitting the interstitial (§6) is **mostly additive** against `player_google_join_test.rb`'s 20 tests rather than destructive: "the bounce page offers the button" still holds because the button stays, and "the bounce page spends nothing" still holds because the handoff is consumed at the callback either way. What it needs is **three new ones** – it fires on a first view, it does *not* fire on a `back_forward` navigation, and it does *not* fire when the handoff has expired (the page renders "That took a moment too long" and there is nothing to submit). That last one is the easiest to get wrong.

P5 · Bug: venue lever misses the interstitial scale <code>PlayerJoinsController</code> 's gate like its sibling	I + test	~20 min	Low. Bites only at <code>SCALE ≥ 3</code> , i.e. exactly on an event day.	Same day
PI – P5 together – what this deck recommends	~55	1½ – 2 days	Two or three pushes. Ship P3 + P4 + P5 first as their own push – they are bugs, they are small, and they are independent of everything else. P1 does not ship without §11.	2 – 3 days
P6 · Delete the interstitial entirely scope <code>request_validation_phase</code> , move the session write to <code>origin_param</code>	~6	2 – 3 days	High, and concentrated: a hand-written replacement for a global security hook that also guards creator sign-in. Needs its own security review, not just a Brakeman pass.	Separate
P7 · "24 waiting on confirmation" for creators explain the gap between claims and audience	~5	~2 hrs	Low. The natural follow-up once P1 makes the number movable.	Follow-up

Deliberately not in this plan

- **Gating anything on confirmation.** §3. The consequence and the remedy are already the same thing.
- **Making `/play/:token` per-session** so its CSRF token is live. The shared cache is a large share of a journey's CPU; trading it for one tap is paying in the thing that carries a venue full of respondents.
- **Allowing `GET` on the OmniAuth request phase.** The config is global – it would re-open creator sign-in to the cross-site initiation class OmniAuth 2.0 closed.
- **Merging the verification mail into the sign-in link.** Different expiries, different proofs, and one of the two is a bearer credential.
- **A bulk mail-out to existing unverified respondents,** until §8's count is known. It is a separate change with its own rate shaping.
- **Re-running a red CI job.** House rule: a red Main is fixed with a new commit, never a re-run.